



Nicolas

Universite de Reims Champagne-Ardenne
Master 1 – Calcul Haute Performance / Imagerie / IA

Annee 2025–2026

1. Introduction

- Definition des regles de Golomb
- Probleme d'optimisation

2. Algorithme

- Branch & Bound
- Strategies de pruning
- BitSet256 et AVX2

3. Architecture

- 5 versions : v1 a v5
- OpenMP, MPI, Pure MPI

4. Propagation $O(\log P)$

- Topologie hypercube
- Algorithme de propagation

5. Resultats

- Scaling OpenMP
- Comparaison v3 vs v4
- Efficacite parallele

6. Conclusion

- Contributions
- Perspectives

Qu'est-ce qu'une Regle de Golomb ?

Definition

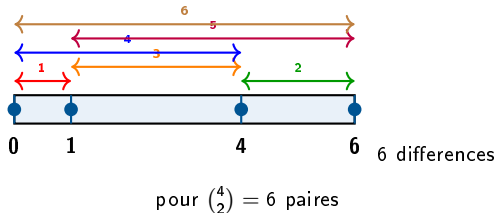
Une **regle de Golomb** d'ordre n est un ensemble de n entiers (marques) tel que toutes les **différences par paires** sont **distinctes**.

Exemple : $G_4 = [0, 1, 4, 6]$

- Differences : $\{1, 2, 3, 4, 5, 6\}$ – toutes uniques
- Longueur : 6 (optimale)

Applications

- Radio-astronomie (antennes)
- Codes correcteurs d'erreurs
- Traitement du signal



Objectif

Trouver la regle de Golomb **optimale** (la plus courte) pour un ordre n donne.

Complexite

- Probleme **NP-difficile**
- Espace de recherche : $O(L^n)$
- G27 trouve en 2014 (calcul mondial)

Borne inferieure (Erdos-Turan) :

$$L \geq \frac{n(n-1)}{2}$$

Longueurs Optimales Connues

Ordre	Long.	Ordre	Long.
4	6	10	55
5	11	11	72
6	17	12	85
7	25	13	106
8	34	14	127
9	44	15	151

Source : OEIS

A003022

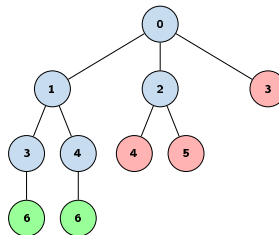
Principe

Exploration systématique de l'arbre avec **elagage** des branches non prometteuses.

Algorithme :

- ❶ Si profondeur = ordre $\rightarrow$ solution trouvée
- ❷ Pour chaque position valide :
 - Placer la marque
 - Vérifier les différences
 - Recursion
 - Backtrack

Arbre de recherche (G4)



Vert = solution, Rouge = elague

1. Borne Gloutonne

Solution gloutonne calculee **avant** la recherche. Placer chaque marque a la plus petite position valide. **Impact** : borne $\sim 80\%$ plus serree

2. Cassure Symetrie

$marks[1] \leq \frac{L}{2}$ Regles reversibles : $[0, a_2, \dots]$
 $\equiv [0, L - a_n, \dots]$ **Impact** : **-50%** espace

3. Pruning Triangulaire

k marques restantes $\Rightarrow \frac{k(k+1)}{2}$ min Si impossible d'ameliorer : elaguer **Impact** : detection precoce

Reduction totale : $\sim 22x$ – G10 : 3.6M noeuds $\rightarrow$ 2.5M explores

Probleme

Verifier si une difference existe :

- `std::set` : $O(\log n)$
- Boucle naive : $O(n^2)$

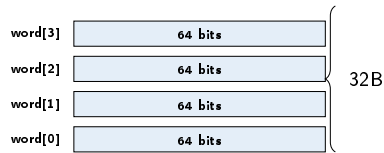
Solution : BitSet256

Structure 256 bits alignee 32 bytes :

- Test/Set en **$O(1)$**
- Cache L1 friendly
- Compatible AVX2

```
struct alignas(32) BitSet256 {  
    uint64_t words[4]; // 256 bits  
    bool test(int bit) const {  
        return words[bit/64] &  
            (1ULL << (bit % 64));  
    }  
    void set(int bit) {  
        words[bit/64] |= (1ULL << (bit%64));  
    }  
};
```

Structure Memoire



Operation	Complexite
-----------	------------

test(bit)	$O(1)$
set(bit)	$O(1)$
collision()	$O(1)$

Principe

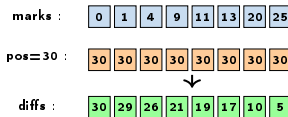
Instructions vectorielles AVX2 :

- 1 Calculer 8 differences simultanément
- 2 Detecter collisions sur 256 bits

```
// 8 differences en 1 instruction
__m256i vpos = _mm256_set1_epi32(pos);
__m256i vmarks = _mm256_loadu_si256(
    (__m256i*)&marks[i]);
__m256i vdiffs = _mm256_sub_epi32(
    vpos, vmarks);

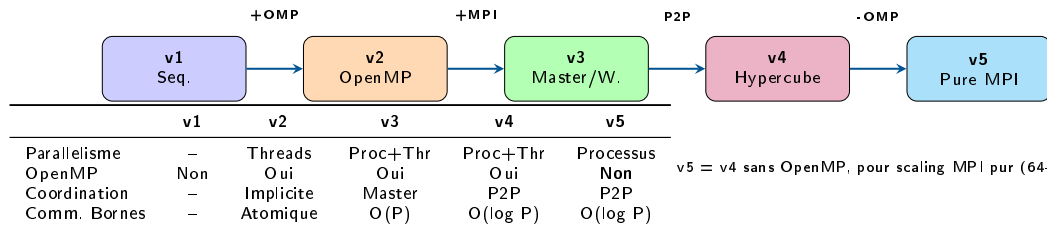
// Detection collision 256-bit
__m256i result = _mm256_and_si256(
    used.data, check.data);
if (!_mm256_testz_si256(result, result))
    return false; // Collision!
```

Vectorisation



Metrique	Valeur
Throughput	8 diff/cycle
Seuil	≥4 marks
Speedup	~2-3x

Vue d'Ensemble des Versions



Comparaison Complete des 5 Versions

Critere	v1	v2	v3	v4	v5
Type	Sequentiel	OpenMP	Master/Worker	Hypercube	Pure MPI
Parallelisme	Aucun	Threads	Proc+Threads	Proc+Threads	Processus
OpenMP	Non	Oui	Oui	Oui	Non
MPI	Non	Non	Oui	Oui	Oui
Coordination	–	Atomique	Master	P2P	P2P
Prop. bornes	–	$O(1)$ local	$O(P)$	$O(\log P)$	$O(\log P)$
Goulot	CPU	Contention	Master	Aucun	Aucun
Scalabilite	1 core	$\sim 64t$	$\sim 16p$	$256+ p$	$512+ p$



v1 – Baseline

- Single-thread + AVX2
- Reference pour speedup
- ~5s pour G11

v2 – OpenMP Tasks

- `#pragma omp task`
- Cutoff depth adaptatif
- $G \leq 7$: $d=1$, $G \leq 9$: $d=2$, $G \geq 10$: $d=3$

Bound Caching

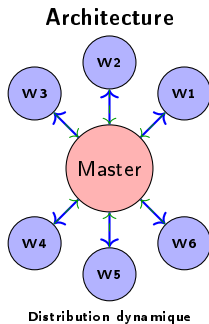
Cache local avec refresh periodique :

```
int localBest = bestLength;
int counter = 0;
// Dans la boucle :
if (++counter % 16384 == 0)
    localBest = atomic_load(&bestLength);
```

Impact : -90% ops atomiques

Cache Alignment

Structures alignees 64 bytes
→ Pas de false sharing



MPI Tags

- TAG_WORK=1 : travail
- TAG_RESULT=2 : resultat
- TAG_BOUND=4 : borne
- TAG_DONE=3 : fin

Profondeur prefixe

$G \leq 8 : d=3$, $G \leq 10 : d=4$
 $G \leq 12 : d=5$, $G > 12 : d=6$

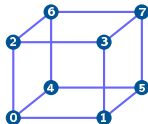
Limite

Master = goulot pour >16 procs

Architecture Decentralisee

- Pas de master → **pas de goulot**
- Distribution statique
- Communication peer-to-peer

Hypercube 8 processus



Avantages v4 vs v3

- ✓ Scalabilite 1000+ procs
- ✓ Communication $O(\log P)$
- ✓ Equilibrage previsible
- ✓ Reproductibilite

Allocation travail

Chaque rang r explore les sous-arbres ou :

$$\text{hash}(\text{subtree}) \bmod P = r$$

Zero communication pour distribution

Principe

Synchroniser la meilleure borne en $\log_2(P)$ rounds.

Algorithme :

- 1 Pour $d = 0$ à $\log_2(P) - 1$:
- 2 Partenaire : $rank \oplus 2^d$
- 3 Echanger les bornes
- 4 Garder le minimum

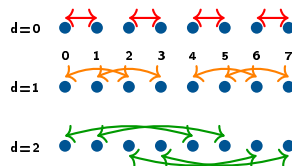
Complexite

Broadcast : $O(P)$

Hypercube : $O(\log P)$

$P=256$: $256 \rightarrow 8$ rounds

Exemple : $P=8$ (3 rounds)



Après 3 rounds : borne
globale

```
void propagateBound(int& best,
                   int rank, int P) {
    int logP = (int)log2(P);
    for (int d = 0; d < logP; ++d) {
        int dist = 1 << d;
        int partner = rank ^ dist;
        // Envoi non-bloquant
        MPI_Request req;
        MPI_Isend(&best, 1, MPI_INT,
                 partner, TAG_BOUND,
                 MPI_COMM_WORLD, &req);

        // Reception
        int pBound;
        MPI_Recv(&pBound, 1, MPI_INT,
                partner, TAG_BOUND,
                MPI_COMM_WORLD,
                MPI_STATUS_IGNORE);
        best = min(best, pBound);
    }
}
```

Operations clés

- $\text{rank} \wedge \text{dist}$: XOR
- MPI_Isend : async
- MPI_Recv : sync

P	Bcast	Hyper
8	8	3
64	64	6
256	256	8
1024	1024	10

Messages par propagation

Cluster Romeo (URCA)

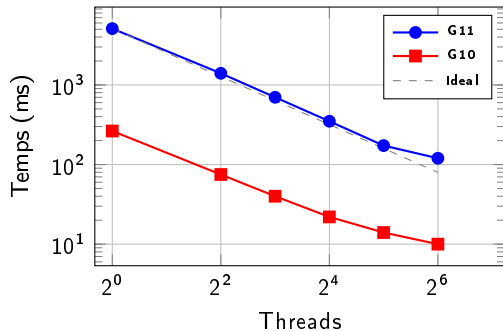
- **CPU** : AMD EPYC 7763
- **Cœurs** : 128/noeud (2.45 GHz)
- **Cache L3** : 768 MB
- **Memoire** : 512 GB/noeud
- **Reseau** : InfiniBand HDR

Configurations testees

- **v1** : 1 thread (baseline)
- **v2** : 1-128 threads
- **v3** : 4-16 ranks $\times$ 4-16 threads
- **v4** : 4-32 ranks $\times$ 8 threads

Problemes testes

G8, G9, G10, G11, G12, G13, G14



Resultats G11

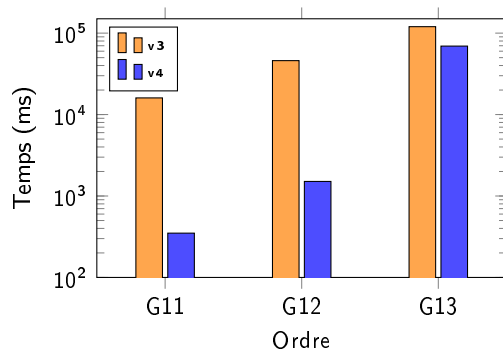
T	Temps	Spdup
1	5099 ms	1x
8	700 ms	7.3x
16	350 ms	14.6x
32	173 ms	29x

Efficacite

$$E = \frac{\text{Speedup}}{P}$$

$$32 \text{ threads : } E = \frac{29}{32} = 91\%$$

Comparaison v3 vs v4



Temps (256 workers)

	v3	v4
G11	16.0 s	0.35 s
G12	45.9 s	1.5 s
G13	120 s	69 s

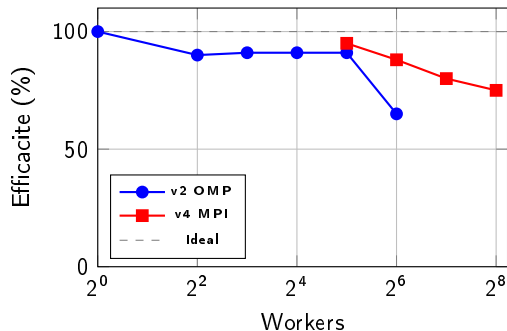
Acceleration v4/v3

G11 : **45x**

G12 : **30x**

v3 : master sature

v4 : $O(\log P)$ scalable



Definition

$$E = \frac{S_p}{p} = \frac{T_1 / T_p}{p}$$

- $E = 100\%$: ideal
- $E > 80\%$: tres bon
- $E > 50\%$: acceptable

Observations

- v2 : $>90\%$ jusqu'a 32t
- v4 : $>75\%$ jusqu'a 256w

Contributions

- ❶ **5 versions** avec progression parallelisation
- ❷ **Optimisations** : pruning, BitSet256, AVX2
- ❸ **Communication $O(\log P)$** via hypercube
- ❹ **Integration HPC** : SLURM, benchmarks

Code

5500+ lignes C++17
58 tests de validation
Structure modulaire

Performances

- G11 : 5.1s → **173ms** (OMP 32t)
- G11 : **350ms** (v4, 256w)
- G12 : **1.5s** (v4, 256w)
- G13 : **69s** (v4, 256w)

Speedup : **29x** (OMP)

Efficacite : **>75%** (256w)

Reduction

Espace recherche : **22x** moins

Améliorations possibles

- **GPU** : CUDA pour collisions
- **BitSet512/1024** : G15+
- **Topologies adaptatives**
- **ML** : stratégies apprises

Limitations

- BitSet256 limite a $L < 256$
- v4 optimal pour $P = 2^k$
- G14+ : plusieurs heures

Questions ?

Merci !

Cluster Romeo – URCA
Master 1 HPC 2025-2026